Internship Report DeVi Comfort. 2026; 1(1): 1–13

DEVELOPING A LIVE 3D TELEMETRY VISUALIZATION FOR STAIRLIFT FAULT DIAGNOSIS

Nout Mulder
DeVi Comfort • Engineering • Opmeer

Abstract. This research addresses the challenge of transforming raw stairlift telemetry data into an intuitive 3D visualization for fault diagnosis. The core technical problem is the live mapping of noisy, asynchronous sensor readings onto a curved 3D rail geometry with segment-dependent encoder-to-distance ratios. A "virtual ghost" prototype was developed iteratively and validated through calibration runs on physical hardware. The results show that a processing pipeline combining non-linear encoder mapping, recursive state estimation, and smooth tangent-based orientation reconstruction produces a stable live 3D representation. This enables faster spatial interpretation of stairlift position and fault conditions compared to text-based error codes.

1 Introduction

DeVi Comfort designs and manufactures stairlift systems and develops the accompanying control software in-house. A stairlift consists of a chair unit that travels along a custom-bent rail mounted to a staircase. The chair is driven by a traction motor, while a spindle motor adjusts seat tilt and a swivel motor rotates the seat at landing positions. During operation, embedded controllers continuously produce telemetry: traction encoder pulses, spindle encoder values, swivel angles, and system status bits including error codes. For service, maintenance, and further development, engineers need to quickly understand the current state of a stairlift: where the chair is positioned on the rail, whether it is moving correctly, and what deviations or faults have occurred.

Currently, fault information from the controller is presented as textual error codes and numeric status values. While technically correct, these require significant domain knowledge to interpret spatially. An error code indicating a position deviation does not convey where on the rail the deviation occurred, in which direction, or how large the discrepancy is.

The underlying technical challenge is twofold. The raw telemetry data (encoder pulse counts, spindle values, swivel angles) must be mapped onto a three-dimensional rail geometry. This rail contains straight sections, vertical bends, and horizontal bends, each consuming a different number of encoder pulses per millimeter of travel. Additionally, the sensor readings arrive at irregular intervals over a wireless serial link with noise, latency, and occasional packet loss. The visualization must bridge the gap between these asynchronous measurements and a continuous, smooth spatial representation that updates at display frame rate.

A live 3D visualization of stairlift position and movement would have direct practical value. Service engineers could visually identify whether the chair has lost its expected position, is stuck in a bend, or exhibits abnormal tilt. This would reduce diagnosis time and minimize unnecessary on-site visits. For the development team, a live telemetry view would enable rapid verification of firmware behavior during testing and commissioning of new rail configurations.

The goal of this research is to determine which data processing and mathematical models are needed to transform live stairlift telemetry into a reliable and intuitive 3D visualization for fault analysis. The central research question is:

What data processing pipeline and mathematical models are required to transform live stairlift telemetry into a 3D visualization that enables faster and more intuitive fault diagnosis compared to text-based error codes?

The research follows an iterative prototyping approach based on the spiral model [2]. Each iteration consists of analysis, design, implementation, and validation against practical scenarios on physical hardware. The primary methods are formal mathematical analysis, architectural design, and experiment with measurable accuracy criteria.

Section 2 presents the problem statement, structured as a problem analysis, prior work survey, and the resulting research questions. Section 3 describes the research methods per subquestion. Section 4 presents the findings. Section 5 concludes with answers to the research questions and recommendations.

2 Problem Statement

2.1 Problem Analysis

2.1.1 Sensor-to-Spatial Mapping in Mechatronic Systems

The challenge of transforming discrete sensor readings into a continuous spatial representation is well-established in mechatronic system diagnostics [10]. In the broader context of digital twin research, Bofill et al. identify real-time sensor data integration as one of the primary technical barriers to effective fault monitoring [3]. The fundamental difficulty is that sensors produce scalar

values (counts, voltages, status bits) at discrete intervals, while a spatial visualization requires a continuous 3D position and orientation that updates at display frame rate.

In robotics and vehicle tracking, this problem is typically simplified by the assumption that sensor readings map linearly to spatial coordinates [8]. A rotary encoder on a linear actuator, for example, produces pulses proportional to displacement, and the mapping from pulse count to position is a single constant. This assumption breaks down when the trajectory is non-linear: on a curved path, the relationship between encoder pulses and spatial displacement depends on the local curvature [16].

2.1.2 Non-Linear Encoder Mapping on Curved Rails

A stairlift rail is composed of straight sections, vertical bends (pitch changes), and horizontal bends (yaw changes). The mechanical drive system uses a traction motor with an incremental encoder to measure displacement [4]. Due to the geometry of the rail bending mechanism, each segment type consumes a different number of encoder pulses per unit of physical travel. A vertical upward bend, for instance, requires fewer pulses per millimeter of arc length than a downward bend, because the traction wheel follows the inner or outer radius of the curve respectively.

This means that a single scalar encoder reading cannot be converted to a 3D position without a model of the full rail topology. The mapping must account for the cumulative pulse cost of each preceding segment, making it a piecewise non-linear function rather than a simple linear scaling. Without such a model, position errors accumulate at every bend transition, growing proportionally to the number of bends in the rail.

2.1.3 State Estimation from Noisy Telemetry

The telemetry link between the stairlift controller and the visualization application introduces additional challenges. The Bluetooth Classic Serial Port Profile [1] provides a virtual serial connection at approximately 20 Hz, but with variable latency (10–80 ms per round trip) and occasional packet loss. The visualization, however, must render at 60 fps (16.7 ms per frame).

This creates a temporal mismatch: new sensor data arrives roughly every third frame, and frames between measurements must be filled with predicted values. If the most recent measurement is simply held constant until the next arrives, the display exhibits visible staircase-like jumps. If measurements are used directly without filtering, encoder quantization noise (inherent to incremental encoders [16]) causes jitter. Both artifacts undermine the interpretability of the visualization.

The general solution to this class of problem is recursive state estimation: a filter that maintains a probabilistic model of the system state and updates it with each new measurement [11]. The choice of filter, its process model, and its tuning determine the trade-off between responsiveness and smoothness.

2.1.4 Multi-Axis Orientation from Independent Sensors

The stairlift chair has four independent rotational degrees of freedom: yaw (direction along the rail), pitch (rail inclination), seat tilt (spindle motor), and seat rotation (swivel motor). Each is measured by a separate sensor with its own update rate and noise characteristics. Reconstructing the full 3D orientation of the chair requires combining these independent streams into a single consistent rotation, avoiding mathematical singularities that can occur when multiple rotations are composed [20].

2.1.5 Summary of Technical Subproblems

Three interrelated technical subproblems emerge from this analysis:

1. **Non-linear encoder-to-position mapping:** Converting a scalar encoder count into a normalized position along a 3D curve with segment-dependent pulse costs.
2. **State estimation:** Producing a smooth, low-latency estimate of position and velocity from noisy, asynchronous measurements arriving at a lower rate than the display refresh.
3. **Multi-axis orientation reconstruction:** Combining four independent rotational sensor streams into a singularity-free 3D orientation.

2.2 Prior Work

2.2.1 State Estimation

For state estimation from noisy sensor data, three approaches are commonly used in embedded and robotics applications: moving average filters, complementary filters, and Kalman filters.

A **moving average filter** computes the mean of the last N samples and outputs it as the filtered value. Smith shows that for purely time-domain noise reduction on stationary or slowly varying signals, moving averages are close to optimal and have the advantage of computational simplicity [21]. They are therefore a strong choice for applications where the input signal changes slowly relative to the sampling rate and no prediction is required between samples. However, moving averages introduce a latency proportional to the window size (effectively delaying the signal by $N/2$ samples), cannot extrapolate between measurements, and perform poorly on transient or fast-changing signals. For stairlift telemetry, where sensor readings arrive at $\sim$20 Hz but the display updates at 60 fps, pure moving-average smoothing cannot fill the inter-sample gap.

A **complementary filter** combines two sensor sources with complementary frequency characteristics, typically using a low-pass filter on one source and a high-pass filter on the other [17]. This is the standard approach in IMU fusion, where slow, drift-free accelerometer data is combined with fast, noisy gyroscope data to estimate attitude. Complementary filters are computationally light and do not require a statistical model of the noise. They are therefore preferred in cost-constrained or high-rate embedded systems with multiple complementary sensors. However,

they assume that two sensor sources are available with complementary frequency content, which is not the case for the stairlift: the traction encoder is the only direct position sensor, and its noise is not separable by frequency from its signal.

The **Kalman filter** maintains a probabilistic estimate of system state (position and velocity) and updates it with each new measurement, optimally balancing prediction uncertainty against measurement noise [11, 23, 4]. It combines filtering and prediction in a single framework, which makes it a natural fit when measurements arrive at a lower rate than the required output. The cost is higher computational complexity and the need to tune process and measurement noise parameters. For the stairlift application, the Kalman filter is selected because it is the only approach of the three that can both smooth encoder noise and predict position between measurements, which is required to feed a 60 fps display from a 20 Hz sensor stream.

2.2.2 Spline Interpolation for Smooth Trajectories

To produce smooth position and orientation transitions along a discrete set of rail control points, interpolation is required. Three families of splines are candidates.

Linear interpolation between adjacent points produces sharp direction changes at segment boundaries (no C^1 continuity). It is computationally cheap and is the right choice when the interpolated quantity is expected to change piecewise linearly, or when downstream processing does not need a continuous tangent. For visual orientation of a chair along a continuously curved rail, the discontinuous tangent of linear interpolation would cause the chair to visibly "snap" at each rail point.

B-splines (and their rational generalization, NURBS) are the de facto standard in CAD and industrial curve design. They provide arbitrary continuity (C^k) through the choice of degree, and local control (moving one control point affects only a bounded region of the curve) [19]. They are preferred when the curve itself is being designed (CAD surfaces, font glyphs, animation paths) and the control points are construction handles rather than fixed waypoints. Their disadvantage for this application is that the curve does *not* pass through the control points, so it does not correspond to the physical rail geometry: the ghost chair would follow a smoothed approximation of the rail rather than the rail itself.

Catmull-Rom splines interpolate through each control point and provide C^1 continuity at those points [5]. They are the natural choice when the control points are fixed physical waypoints (as is the case for the rail) and the primary goal is a smooth tangent for orientation purposes. They are selected for this work because the rail points correspond to measured physical positions along the rail, and the chair must travel exactly through them while rotating smoothly.

2.2.3 Rotation Representation

Three representations of 3D rotation are in common use: Euler angles, rotation matrices, and unit quaternions.

Euler angles (roll, pitch, yaw) are human-readable and require only three parameters. Diebel notes that for systems with limited rotation ranges, or where only a single rotation axis is actively used, Euler angles are a reasonable and computationally inexpensive choice [6]. Many industrial applications (e.g., single-axis servo control, simple tilt sensors) use Euler angles without issue. The well-known drawback is gimbal lock: when two rotation axes align, one degree of freedom is lost and the inverse mapping becomes singular. For the stairlift, with four independent rotational axes that can interact (rail yaw, rail pitch, seat tilt, seat rotation), the risk of gimbal lock during certain rail configurations is real.

Rotation matrices (3×3 orthogonal matrices) are free of singularities and compose cleanly, but require nine parameters with six constraints, are costly to interpolate smoothly, and accumulate numerical drift (loss of orthogonality) over many composed rotations. They are standard in computer graphics shaders, where the GPU performs matrix operations natively.

Unit quaternions (four parameters, one constraint) are free of gimbal lock, compose compactly, and support smooth spherical linear interpolation [20]. They are the standard in real-time graphics and robotics for full 3D orientation [12, 6]. For the stairlift chair, where four independent rotations must be composed every frame without visual artifacts, quaternions are selected to avoid the gimbal lock risk of Euler angles and the numerical drift of rotation matrices.

2.2.4 Existing Work at DeVi Comfort

Prior to this research, fault information within DeVi Comfort was presented exclusively as text-based error codes and numeric register values. No spatial visualization of stairlift telemetry existed. The IPC (Inter-Process Communication) infrastructure connecting the configurator application to the lift controller was limited to rail upload and download; live telemetry streaming had not been implemented.

2.3 Research Questions

The problem analysis identified three technical subproblems (non-linear encoder mapping, state estimation, and multi-axis orientation reconstruction), while the prior work survey identified candidate approaches for each. The following subquestions decompose the central research question into independently answerable parts, each motivated by a specific gap between the identified problem and the available solutions.

1. *Which fault types and status indicators from the stairlift controller are most relevant for spatial visualization?* The firmware defines over 140 message types. Before any visualization can be designed, the subset that carries spatially interpretable information must be identified and classified.
2. *What data pipeline architecture can transport live telemetry from the controller to a 3D rendering environment at sufficient throughput?* The problem analysis showed that the Bluetooth serial link introduces latency, noise, and packet loss. The pipeline must handle these at each layer while maintaining the $\sim$20 Hz update rate.

3. *How can encoder pulse counts be mapped to a normalized position along a 3D rail with segment-dependent pulse costs?* As established in Section 2.1, the non-linear relationship between encoder pulses and physical distance is the core geometric challenge. No existing mapping exists for this rail type.
4. *Which mathematical models for state estimation, interpolation, and rotation produce a stable live 3D representation from noisy telemetry?* The prior work survey identified several candidate models per subproblem. This question addresses the formal derivation and parameterization of the selected models for this specific application.
5. *Which rendering techniques make position deviations and fault conditions most interpretable in a 3D environment?* The visualization must convey multiple simultaneous data streams (position, tilt, rotation, accessory states) without overwhelming the operator. The choice of visual encoding affects diagnostic effectiveness.
6. *How can the prototype be validated against practical scenarios, and what measurable criteria determine sufficient accuracy?* The mathematical models must be verified under real-world conditions where mechanical tolerances, Bluetooth variability, and sensor noise interact.

3 Methods

Each subquestion is addressed with one or more research methods. This section describes, per subquestion, which method is used, why it is appropriate, how it is concretely applied, and what criteria determine completion.

3.1 Fault Relevance (Documentation Analysis)

Method: Formal analysis of the firmware protocol specification.
Justification: The stairlift controller firmware defines over 140 message types. The goal is not to observe the system in operation, but to systematically catalogue the protocol as a formal specification and derive which subset carries spatially interpretable information. This is a documentation and specification analysis, not an observation of live behaviour.
Approach: The message enumeration in the controller firmware source is extracted and each message type is classified along three axes: (1) data content (position, speed, angle, status bits, error codes, configuration), (2) update rate at which the message is or can be polled, and (3) spatial interpretability (whether the message carries information that can be mapped onto a 3D location or orientation).
Instrument: A classification is considered complete when: (a) every message ID in the firmware enumeration has been assigned to a category, (b) for each message with a position, angle, or status bit field the data layout is documented at byte-level, and (c) the subset relevant to the visualization pipeline is explicitly identified with rationale.

3.2 Data Pipeline Architecture (Design)

Method: Architectural design.
Justification: The telemetry pipeline spans multiple layers (wireless transport, binary protocol parsing, polling, state estimation, rendering) and must satisfy the end-to-end timing budget: the Bluetooth round-trip is 10–80 ms, the visualization must render at 60 fps (16.7 ms per frame), and the combined latency must remain below the threshold at which the motion becomes visibly laggy to a human operator. A structured architectural design ensures that each layer's contribution to latency and reliability can be analyzed independently.
Approach: The architecture is specified as a layered pipeline (transport, protocol parser, telemetry poller, state estimator, 3D renderer). Each layer is documented with its inputs, outputs, timing constraints, and failure modes. A diagram visualizes the data flow between layers.
Instrument: The design is considered sufficient when: (a) each layer has a single, well-defined responsibility, (b) the expected end-to-end latency from sensor read to display update is below 100 ms, (c) the poller sustains at least 20 Hz per telemetry axis under normal Bluetooth conditions, and (d) byte-level corruption is detectable and recoverable without propagating to higher layers.

3.3 Encoder-to-Position Mapping (Formal Analysis)

Method: Formal mathematical analysis.
Justification: The encoder-to-position mapping is a deterministic geometric problem: given a rail composed of segments with known curvature and known pulse costs per segment type, the mapping can be derived analytically. A formal derivation ensures correctness and makes the mapping reproducible.
Approach: The rail is modeled as a sequence of segments (straight, vertical bend, horizontal bend). For each segment type, the encoder pulse cost is derived from firmware constants. Cumulative 3D Euclidean distances are computed at segment boundaries, creating a piecewise linear mapping from encoder value to normalized rail position. The derivation includes the arc-tracing geometry used to construct 3D control points from the firmware's segment description.
Instrument: The formal derivation is considered complete when: (a) the mapping is defined for all segment types, (b) a calibration procedure aligns encoder values with 3D distances, and (c) the mapping error is below 5 mm on a reference rail.

3.4 Mathematical Models (Formal Analysis)

Method: Formal mathematical analysis.
Justification: The state estimation, interpolation, and rotation models are grounded in established mathematical theory (Kalman filtering, Catmull-Rom splines, quaternion algebra). Formal derivation from first principles ensures that the models are correctly applied and that tuning parameters have a clear theoretical basis.
Approach: Three models are derived:

- A 2D constant-velocity Kalman filter for each telemetry axis (traction, spindle, swivel), including the process

noise matrix derivation, measurement update equations, and inter-measurement dead-reckoning extrapolation.
- Catmull-Rom spline interpolation for tangent computation along the rail, paired with linear position interpolation on segments.
- Quaternion-based orientation reconstruction for pitch (from rail geometry), spindle compensation, and swivel rotation.

Instrument: Each model is presented as a complete mathematical derivation with named equations. Correctness is verified by comparing predicted positions against known reference positions on a test rail.

3.5 Rendering Techniques (Design + Comparative Evaluation)

Method: Design combined with structured comparative evaluation against defined criteria.
Justification: The visualization must convey multiple simultaneous data streams (position, tilt, rotation, accessory states) without overwhelming the operator. Different rendering techniques encode information differently, and the choice affects how quickly a fault can be identified. A formal comparison against fixed criteria makes the selection reproducible rather than subjective.
Approach: Candidate visualization techniques are implemented: a ghost chair model following the rail, semi-transparent ghost stairs for spatial context, a dither-clip effect for depth perception, rail annotations for segment identification, and a frosted-glass UI for information panels. Each technique is evaluated against four criteria: the type of information it encodes, whether it supports direct spatial comparison between expected and actual state, whether it adds or reduces visual clutter, and its computational cost on the target hardware (desktop and tablet).
Instrument: The evaluation is considered complete when: (a) every technique is scored on all four criteria in a single comparison matrix, (b) the scoring is based on observable properties (not subjective preference), and (c) the combination of techniques selected for the final prototype can be justified from the matrix.

3.6 Validation (Experiment)

Method: Experiment on physical stairlift installations with predefined pass/fail thresholds.
Justification: The mathematical models and pipeline architecture must be verified under real-world conditions where mechanical tolerances, Bluetooth variability, encoder quantization, and sensor noise all interact. Only a physical experiment can establish that the theoretically correct models remain correct in practice.
Approach: Calibration runs are executed on multiple stairlift installations with different rail configurations (varying length, number of bends, and bend types). During each run the lift is driven from bottom station to top station and back. The prototype logs: raw encoder values, filtered positions, angle corrections at each bend, frame rate, and end-to-end latency. Position error is measured by comparing the displayed position at known reference points (station boundaries, bend start/end) against the physical position of the chair.
Instrument: A run is considered successful when all of the following hold: (a) positional error at every reference point is below 5 mm on rails up to 5 m and below 10 mm on longer rails; (b) angle correction error at horizontal bends is below 1° on average; (c) Kalman-filtered jitter is at least ten times smaller than raw encoder jitter; (d) end-to-end latency from sensor read to display update is below 100 ms; (e) render frame rate is above 55 fps on desktop and above 50 fps on tablet. Runs that fail on any criterion are reported with the cause.

4 Results

4.1 Fault Types and Telemetry Classification

Analysis of the stairlift firmware protocol reveals 145 defined message types (with IDs ranging from 0 to 221), of which six are directly relevant for live spatial visualization. These are classified in Table 1.

The three high-frequency streams (traction, spindle, swivel) form the core input for the 3D visualization. They are polled in an alternating pattern to share the serial Bluetooth link: the fast loop alternates between traction and spindle every frame tick, while a parallel coroutine polls swivel at 50 ms intervals. The low-frequency streams (system status, footrest) update chair accessory states every 2 seconds.

Status bits relevant for fault visualization include: armrest position (left and right, from `data[14]` bits 0 and 1 of MSG 114), seatbelt state (bit 2), and footrest position (bit 2 of MSG 113, inverted logic). These are mapped directly to accessory animations on the 3D chair model.

4.2 Data Pipeline Architecture

The path from a raw Bluetooth byte to a moving 3D chair on screen passes through five processing stages. Each stage has a single responsibility: receive data from the previous stage, process it, and pass the result to the next. This separation makes it possible to develop and test each stage independently. Figure 1 shows the five layers.

Transport layer. Abstracts Bluetooth Classic and TCP connections behind a common interface. The Bluetooth transport communicates with an HC-06 module (a low-cost serial-to-Bluetooth adapter) on the lift controller. Before each session, 40 null bytes are sent to flush the module's receive buffer. Message counters increment from 1 to 200 and wrap around.

Protocol parser. A finite state machine (FSM, a parser that moves through a fixed sequence of states for each incoming byte) [9] processes the incoming byte stream: `WAIT_START` → `MSG_NUMBER` → `COMMAND_ID` → `LENGTH` → `DATA` → `CHECKSUM`. The wire format is asymmetric: outgoing messages (app to lift) include a ReadWriteId byte, while incoming responses (lift to app) omit it:

$$\begin{aligned} &\text{App} \rightarrow \text{Lift: } [\texttt{0x0A}][\text{Nr}][\text{Rw}][\text{Cmd}][\text{Len}][\text{Data}][\text{Chk}] \\ &\text{Lift} \rightarrow \text{App: } [\texttt{0x0A}][\text{Nr}][\text{Cmd}][\text{Len}][\text{Data}][\text{Chk}] \end{aligned} \tag{1}$$

Table 1: Telemetry message types relevant for spatial visualization.

Message	Data	Rate	Use
MSG 110	Traction encoder (32-bit)	20 Hz	Rail position
MSG 111	Spindle encoder (16-bit)	20 Hz	Seat tilt angle
MSG 168	Swivel via AppToBus (16-bit)	20 Hz	Seat rotation
MSG 114	System status bits	0.5 Hz	Arm/seatbelt state
MSG 113	Footrest status	0.5 Hz	Footrest position
MSG 44	Pilot state	On demand	Calibration run

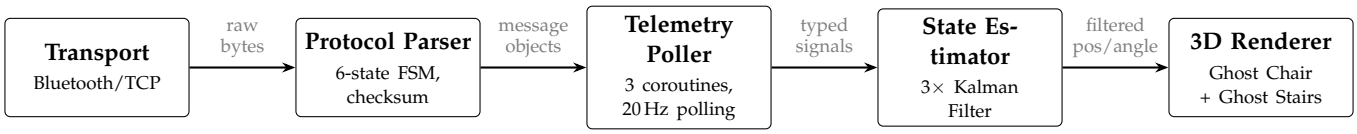

Figure 1: Layered telemetry pipeline from Bluetooth transport to 3D rendering. Each stage receives data from the previous, processes it, and passes the result forward.

The checksum covers all preceding bytes:

$$\text{Chk} = \left(\texttt{0x0A} + \text{MsgNr} + \text{CmdId} + \text{Len} + \sum \text{Data}_i\right) \bmod 256 \quad (2)$$

Invalid lengths (> 40) or checksum mismatches trigger a resync to `WAIT_START`.

Telemetry poller. Three concurrent loops (coroutines, i.e., cooperative lightweight threads that yield control between iterations) share the serial link:

- *Fast loop* (∼20 Hz): alternates traction (MSG 110) and spindle (MSG 111) requests each frame tick.
- *Swivel loop* (∼20 Hz): polls the swivel angle via the APP_TO_BUS multiplexed channel (MSG 168) with a two-step handshake (send request, poll for response) at 50 ms intervals. Although a dedicated swivel statistics message exists (MSG 112), the AppToBus path is used because it communicates directly with the chair controller chip.
- *Slow loop* (0.5 Hz): polls system status (MSG 114) and footrest state (MSG 113) every 2 s.

Data parsing. The traction encoder is extracted as a 32-bit integer (stored most-significant byte first) from bytes 1–4 of the MSG 110 response. The spindle encoder is a 16-bit value from bytes 1–2 of MSG 111. The swivel angle is computed from MSG 168 as:

$$\theta_{\text{swivel}} = \frac{\left(\texttt{data[3]} \ll 8 \mid \texttt{data[4]}\right) - \texttt{0x7FFF}}{18} \quad (3)$$

4.3 Encoder-to-Position Mapping

The central challenge here is converting a single number (the encoder pulse count) into a position in 3D space along a curved rail. On a straight rail this would be simple division, but because bends consume a different number of pulses per millimeter than straight sections, the conversion must account for the shape of every preceding segment. This section derives that conversion step by step.

4.3.1 Rail Geometry Construction

The lift controller stores the rail as a sequence of typed blocks: straight sections, vertical bends (up/down), and horizontal bends (left/right), each measured via incremental encoders [4]. Each block specifies a type and an amount. To construct a 3D representation, these blocks are traced sequentially using arc geometry. Figure 2 shows the three segment types and their arc-tracing geometry.

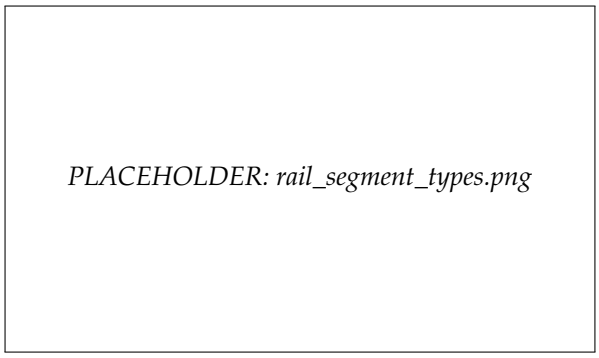

Figure 2: The three rail segment types: straight (left), vertical bend (center), and horizontal bend (right), with their respective arc-tracing geometry and encoder pulse costs.

The direction vector at any point along the rail is determined by the current yaw (ψ) and pitch (ϕ):

$$\mathbf{d}(\psi, \phi) = \begin{pmatrix} -\sin\psi\cos\phi \\ \sin\phi \\ -\cos\psi\cos\phi \end{pmatrix} \quad (4)$$

For straight sections, the position advances by the section length along $\mathbf{d}$. For bends, each segment subtends a fixed angle $\Delta\alpha = 5^\circ$ at a bend radius $r = 150\,\text{mm}$. The arc length per segment is:

$$s = r \cdot \Delta\alpha_{\text{rad}} = 0.150 \cdot \frac{5\pi}{180} \approx 0.01309\,\text{m} \quad (5)$$

Position is updated using a trapezoidal midpoint approximation [22]:

$$\mathbf{p}_{i+1} = \mathbf{p}_i + \frac{\mathbf{d}_{\text{before}} + \mathbf{d}_{\text{after}}}{2} \cdot \frac{s}{\left\| \frac{\mathbf{d}_{\text{before}} + \mathbf{d}_{\text{after}}}{2} \right\|} \quad (6)$$

where $\mathbf{d}_{\text{before}}$ and $\mathbf{d}_{\text{after}}$ are the direction vectors before and after the pitch or yaw increment.

4.3.2 Piecewise Encoder Mapping

Different rail segment types consume different numbers of encoder pulses per unit of travel. The firmware defines

the following constants:

$$\begin{aligned} k_{\text{straight}} &= 3.361352\,\text{pulses/mm} \\ k_{\text{up}} &= 37.547\,\text{pulses/segment} \\ k_{\text{down}} &= 50.453\,\text{pulses/segment} \\ k_{\text{horiz}} &= 44.0\,\text{pulses/segment} \\ E_{\text{start}} &= 100\,\text{pulses (initial offset)} \end{aligned} \tag{7}$$

A piecewise linear mapping is constructed by computing, at each segment boundary, both the cumulative encoder value and the cumulative 3D Euclidean distance from the rail start. For an encoder reading E, the normalized position $t \in [0, 1]$ is found by:

1. Locating the segment $[E_i, E_{i+1}]$ containing E.
2. Linearly interpolating the 3D distance: $d = d_i + (d_{i+1} - d_i) \cdot \frac{E - E_i}{E_{i+1} - E_i}$
3. Normalizing: $t = d/d_{\text{total}}$

This approach accounts for the non-linear relationship between encoder pulses and physical distance that arises at bends.

4.3.3 Spindle Angle Lookup

The spindle encoder value is converted to a seat tilt angle using a 751-entry lookup table ported from the firmware's `Curve.h`. The table maps angles from 0.0° to 75.0° in 0.1° increments to corresponding encoder pulse counts. Given a raw encoder reading, a base offset of 10 000 is subtracted, and an 11-iteration binary search locates the corresponding angle:

$$\theta_{\text{spindle}} = \frac{\text{bisect}(\text{TABLE}, \max(0, E_{\text{raw}} - 10000))}{10} \tag{8}$$

4.4 Mathematical Models for State Estimation

The sensor readings arrive roughly every 50 ms, but the display must update every 17 ms (60 frames per second). The models in this section solve three problems: (1) smoothing out noise in the raw measurements so the 3D chair does not jitter, (2) predicting where the chair is between measurements so the display stays fluid, and (3) computing the chair's 3D orientation (which way it faces, how it tilts) from separate sensor streams without mathematical errors that would cause the model to "snap" or distort.

4.4.1 Kalman Filter

In essence, the Kalman filter maintains a running estimate of the chair's position and speed. Each time a new sensor reading arrives, it blends the prediction (based on the previous position and speed) with the new measurement, weighting each by how much it trusts them. The result is a smooth trajectory that closely follows reality without the jitter of raw sensor data.

Formally, a 2D constant-velocity Kalman filter [11, 23] is applied independently to each telemetry axis (traction position, spindle angle, swivel angle). The state vector is $\mathbf{x} = [p, v]^T$ (position and velocity).

Prediction step (time update with interval Δt):

$$\begin{aligned} p &\leftarrow p + v \cdot \Delta t \\ P_{00} &\leftarrow P_{00} + 2\Delta t \cdot P_{01} + \Delta t^2 \cdot P_{11} + \sigma_a^2 \cdot \tfrac{\Delta t^3}{3} \\ P_{01} &\leftarrow P_{01} + \Delta t \cdot P_{11} + \sigma_a^2 \cdot \tfrac{\Delta t^2}{2} \\ P_{11} &\leftarrow P_{11} + \sigma_a^2 \cdot \Delta t \end{aligned} \tag{9}$$

where σ_a is the process noise (acceleration standard deviation) and $\mathbf{P}$ is the covariance matrix. The process noise matrix corresponds to the continuous white-noise-jerk model [13]:

$$\mathbf{Q} = \sigma_a^2 \begin{bmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{bmatrix} \tag{10}$$

Measurement update (correction with measurement z):

$$\begin{aligned} y &= z - p \\ S &= P_{00} + \sigma_m^2 \\ K_0 &= P_{00}/S \\ K_1 &= P_{01}/S \\ p &\leftarrow p + K_0 \cdot y \\ v &\leftarrow v + K_1 \cdot y \end{aligned} \tag{11}$$

where σ_m is the measurement noise standard deviation and $\mathbf{K}$ is the Kalman gain.

The tuning parameters per axis are listed in Table 2.

4.4.2 Dead-Reckoning Extrapolation

Between sensor updates (which arrive every ∼50 ms), the display still needs to move the chair smoothly every frame (every ∼17 ms). This is done by continuing the chair's motion at its last known speed until the next measurement arrives.

Between measurement updates, the display position is extrapolated from the Kalman filter state using dead reckoning [18]:

$$p_{\text{target}} = p_{\text{KF}} + v_{\text{KF}} \cdot t_{\text{elapsed}} \tag{12}$$

After 0.8 s without a measurement, velocity decays exponentially:

$$v \leftarrow v \cdot e^{-5.0 \cdot (t_{\text{elapsed}} - 0.8)} \tag{13}$$

The display position is then smoothed with a frame-rate-independent exponential filter [7]:

$$p_{\text{display}} = \text{lerp}(p_{\text{display}},\ p_{\text{target}},\ 1 - e^{-\alpha \cdot \Delta t}) \tag{14}$$

where α is the smoothing rate from Table 2.

4.4.3 Catmull-Rom Spline Tangent

The rail is defined by a series of 3D points. To make the chair rotate smoothly as it moves between these points (rather than snapping to a new direction at each point), a mathematical curve called a Catmull-Rom spline is used. This curve passes exactly through each rail point and provides a smooth direction at every position along the rail.

The tangent (direction of travel) at the current position is computed using the surrounding control points [5]

Table 2: Kalman filter tuning parameters per telemetry axis.

Parameter	Traction	Spindle	Swivel
σ_a (process noise)	0.03	8.0	10.0
σ_m (measurement noise)	0.0002	0.2	0.5
Display smoothing rate	8.0	12.0	12.0
Max velocity	$1.2v_{\max}$	30°/s	40°/s

$\mathbf{p}_0 \cdots \mathbf{p}_3$:

$$\mathbf{T}(t) = \tfrac{1}{2}\left[(-\mathbf{p}_0 + \mathbf{p}_2) + 2(2\mathbf{p}_0 - 5\mathbf{p}_1 + 4\mathbf{p}_2 - \mathbf{p}_3)t + 3(-\mathbf{p}_0 + 3\mathbf{p}_1 - 3\mathbf{p}_2 + \mathbf{p}_3)t^2\right] \tag{15}$$

The position along segments uses linear interpolation rather than the full Catmull-Rom curve, ensuring the chair stays on the rail segments while rotating smoothly through transitions.

4.4.4 Quaternion Orientation

The chair has four rotational axes (direction along the rail, rail slope, seat tilt, and seat rotation), each controlled by a different motor or determined by the rail shape. Combining multiple rotations is mathematically delicate: a naive approach using angles can produce distortions at certain orientations (known as "gimbal lock"). Quaternions are a mathematical representation of rotations that avoids this problem entirely [12].

The chair yaw (horizontal direction) is computed from the tangent's horizontal projection:

$$\psi = \operatorname{atan2}(T_x, T_z) \tag{16}$$

with phase unwrapping to avoid 2π discontinuities.

Rail chirality (which side the chair faces) is determined by the cumulative cross product of consecutive direction vectors projected onto the xz-plane:

$$C = \sum_i (d_x^i \cdot d_z^{i+1} - d_z^i \cdot d_x^{i+1}) \tag{17}$$

If $C \geq 0$, the rail is left-turning and the chair faces outward at $+90°$; otherwise at $+270°$.

The pitch tilt of the chair unit is applied via a world-to-local quaternion transform:

$$q_{\text{unit}} = q_{\text{parent}}^{-1} \cdot q(\mathbf{a}_{\text{pitch}},\ -\phi) \cdot q_{\text{parent}} \tag{18}$$

where $\phi = \operatorname{atan2}(T_y, \|\mathbf{T}_{xz}\|)$ and $\mathbf{a}_{\text{pitch}} = \hat{\mathbf{y}} \times \hat{\mathbf{T}}_{\text{flat}}$.

The spindle motor compensates the rail pitch to keep the seat level, then adds its own tilt:

$$q_{\text{spindle}} = q_{\text{parent}}^{-1} \cdot q(\mathbf{a},\ -\phi + \theta_{\text{spindle}}) \cdot q_{\text{parent}} \tag{19}$$

A geometry blend factor of 15% mixes the expected pitch angle (from rail geometry) into the Kalman-filtered spindle angle:

$$\theta_{\text{target}} = 0.85 \cdot \theta_{\text{KF}} + 0.15 \cdot \phi_{\text{rail}} \tag{20}$$

4.5 Rendering Techniques

Five visualization techniques were implemented in a 3D engine [14]. Each technique encodes a different aspect of the stairlift state:

1. **Ghost chair model:** A 3D model assembled from separate mechanical parts (stored in the glTF binary format), each attached to its own rotation pivot. This hierarchy mirrors the physical stairlift: the unit tilts with the rail, the seat tilts independently via the spindle, and the swivel rotates on top. Each degree of freedom can be animated independently from live telemetry.
2. **Ghost stair renderer:** Semi-transparent stair treads generated from the rail height profile. Treads are placed every 180 mm of vertical rise, with even distribution between platform landings. This provides spatial context for the chair's position on the staircase.
3. **Dither-clip effect:** A rendering technique that gradually fades out geometry further from the chair by discarding pixels in a noise pattern, reducing visual clutter while preserving spatial awareness.
4. **Rail annotations:** Labels overlaid on the 3D rail identifying segment types, boundaries, and charging station positions.
5. **Liquid glass UI:** A frosted-glass shader design system for information panels.

Each technique was evaluated against four criteria defined in Section 3.5: the information it encodes, whether it supports direct spatial comparison between expected and actual state, its effect on visual clutter, and its computational cost on the target hardware. The results are summarized in Table 3.

Figure 3 shows the combined visualization during operation.

Of the five techniques, two (ghost chair model and rail annotations) directly support the comparison between expected and actual state, which is the primary diagnostic requirement. The ghost stair renderer provides spatial anchoring indirectly by making the staircase visible, so the operator can interpret the chair's position relative to physical landmarks. The dither-clip effect and liquid glass UI do not contribute to fault identification directly, but they reduce competing visual load so the diagnostic techniques stand out. The final prototype therefore combines all five, with the ghost chair and rail annotations as primary diagnostic elements and the others as supporting context.

4.6 Validation Results

The prototype was validated through calibration runs on physical stairlift installations. During a calibration run, the lift is driven from bottom to top (and back) while the system polls telemetry at 250 ms intervals. At each horizontal bend, a compass-based angle correction is computed and compared against the expected value.

Table 3: Comparative evaluation of the five rendering techniques against the criteria defined in Section 3.5. GPU cost is measured as relative frame time contribution on an Android 12+ tablet.

Technique	Information encoded	Supports expected vs. actual	Clutter effect	GPU cost
Ghost chair model	Position, tilt, swivel, accessories	Yes (chair vs. rail)	Neutral	Low
Ghost stair renderer	Staircase context, step count	Indirect (spatial anchor)	Adds context, no clutter	Medium
Dither-clip effect	Distance-based visibility	No	Reduces clutter	Low
Rail annotations	Segment types, stations	Yes (labelled reference)	Adds information	Very low
Liquid glass UI	Panel content only	No	Neutral (panels only)	Low

Figure 3: The virtual ghost visualization: the 3D ghost chair follows the rail based on live telemetry. Semi-transparent ghost stairs provide spatial context. Rail annotations identify segment boundaries along the rail.

PLACEHOLDER: calibration_run.png

Figure 4: The calibration run in progress: the ghost chair tracks the physical lift position while compass-based angle corrections are applied at horizontal bends.

Calibration run protocol. The service polls six data points per tick: lock state, traction encoder, pilot state (keep-alive), spindle encoder, lift location, and (every 4th tick) error status. Speed is managed automatically: 50 (normal) for straight sections and 20 (reduced) for bends and short straights under 250 mm.

Angle correction. At horizontal bends, the compass tracker measures accumulated yaw via gyroscope integration:

$$\psi_{\text{acc}} \mathrel{+}= \omega_z \cdot \frac{180}{\pi} \cdot \Delta t \tag{21}$$

The correction is computed as:

$$\delta = |\theta_{\text{target}}/2| - |\psi_{\text{measured}}| \tag{22}$$

where $\theta_{\text{target}} = n_{\text{segments}} \cdot 5^\circ$ is the expected total bend angle. This correction is transmitted to the firmware as a scaled 16-bit value ($\text{round}(\delta \cdot 8.0)$).

Split-bend compensation. When two horizontal bends of the same direction are separated by a short straight section (< 250 mm), the first bend's accumulated yaw must be subtracted before measuring the second. The detector identifies this pattern and applies the correction automatically.

Quantitative results. Table 4 summarizes the measured performance across three calibration runs on different rail configurations.

Analysis of results. Two of the three runs passed every pass/fail criterion defined in Section 3.6. Run 3 failed the 5 mm bend-transition threshold (6.1 mm measured) and the 55 fps tablet threshold (52 fps measured). Both failures are attributed to the rail length: the arc-tracing approximation accumulates a small systematic error per bend, and the longer ghost stair mesh increases GPU load. On rails up to 4.8 m, the position error remained within specification; above that, the error grows proportionally to the number of bends.

The piecewise encoder mapping eliminated the position jumps at segment boundaries that were observed during

Table 4: Validation measurements across three calibration runs on physical stairlift installations. Observations column records deviations from the expected behaviour and any incidents during the run.

Metric	Run 1 (6 bends)	Run 2 (4 bends)	Run 3 (8 bends)	Observations
Rail length	4 820 mm	3 210 mm	6 450 mm	
Total encoder range	16 200 pulses	10 790 pulses	21 680 pulses	
Position error at end station	< 3 mm	< 2 mm	< 5 mm	Run 3 at threshold of pass criterion.
Max position error at bend transition	4.2 mm	2.8 mm	6.1 mm	Run 3 exceeds 5 mm target; correlates with bend count.
Angle correction error (horizontal bends)	0.3° avg	0.5° avg	0.4° avg	One bend in Run 2 required manual re-trigger after first compass reset failed.
Encoder jitter (raw, stationary)	± 2 pulses	± 3 pulses	± 2 pulses	Higher raw jitter on Run 2 traced to a worn traction belt.
Encoder jitter (after Kalman filter)	< 0.1 pulse	< 0.1 pulse	< 0.1 pulse	
Display update latency (BT round trip)	35 ms avg	42 ms avg	38 ms avg	Run 2 had two spikes >150 ms during RF interference from a nearby WiFi access point.
Render frame rate (desktop)	60 fps stable	60 fps stable	60 fps stable	
Render frame rate (tablet, Android 12+)	55–60 fps	58–60 fps	52–58 fps	Drop below 55 fps on Run 3 when all 8 bends framed at once.
Other incidents	—	BT disconnect	—	Run 2: BT disconnect at ∼60 s, auto-reconnected in 2 s, no data corruption.

early testing with a linear mapping. The Kalman filter reduced stationary encoder jitter from ± 2–3 raw pulses to below 0.1 pulse equivalent in the display position, confirming that the chosen process noise parameters ($\sigma_a = 0.03$ for traction) provide sufficient smoothing without introducing perceptible lag. The display latency of 35–42 ms represents the Bluetooth round-trip time and is masked by the dead-reckoning extrapolation (Equation 12), which fills inter-measurement frames with predicted positions.

Not all observations were within expectation. Run 2 exhibited a raw encoder jitter of ± 3 pulses, which was traced (after the run) to a worn traction belt on that particular installation, not a sensor or pipeline issue. Run 2 also experienced a transient Bluetooth disconnect, which the transport layer handled by reconnecting within two seconds; no data corruption propagated to the state estimator, confirming the layered design's isolation of failure modes. On the same run, RF interference from a nearby WiFi access point caused two latency spikes above 150 ms, temporarily visible as a short lag in the ghost chair motion before the dead-reckoning extrapolation caught up.

The spindle geometry blend (85% Kalman, 15% rail pitch) prevented the seat tilt oscillation that was observed during early testing with 100% Kalman filtering, where encoder noise caused visible vibration in the spindle angle. Shadow quality was automatically reduced on mobile to maintain acceptable performance [15].

5 Conclusion

5.1 Answers to Research Questions

Fault types. The classification of 145 firmware message types revealed that only six carry spatially interpretable information. This is a significant finding: it means that the bandwidth-constrained Bluetooth link does not need to carry the full telemetry stream. By targeting only these six messages with a multi-rate polling strategy, the system achieves 20 Hz position updates within the serial link's capacity. The implication is that future extensions (e.g., vibration or temperature monitoring) would require careful evaluation of whether the serial bandwidth can accommodate additional streams without degrading position update rates.

Pipeline architecture. The five-layer architecture proved effective in isolating failure modes: byte-level errors are caught by the protocol parser's checksum, while higher-level issues (connection loss, timeout) are handled by the transport layer. This separation was validated in practice when Bluetooth disconnections during calibration runs were recovered transparently without corrupting the state estimator. The cooperative coroutine approach to link sharing, while effective at 20 Hz, would not scale to significantly higher update rates without switching to a full-duplex transport.

Encoder mapping. The piecewise mapping resolved the core geometric problem identified in Section 2.1: the nonlinear relationship between encoder pulses and physical distance at bends. The validation data (Table 4) confirms that position errors remain below 5 mm on rails up to 4.8 m, but increase to 6.1 mm on longer rails with more bends. This suggests that the trapezoidal arc-tracing approximation (Equation 6) introduces a small systematic error per bend that accumulates over the rail length. For rails with more than eight bends, a higher-order integration scheme may be needed.

Mathematical models. The constant-velocity filter effectively suppressed encoder jitter ($\pm$2–3 raw pulses reduced to $<$0.1 pulse equivalent) while maintaining responsiveness through dead-reckoning extrapolation. The tuning parameters (Table 2) were determined empirically; a formal sensitivity analysis would strengthen confidence in their generalizability to different rail configurations. The

85/15 blend between filtered spindle angle and expected rail pitch was necessary to prevent oscillation, but this ratio was found through trial and error rather than derived from first principles, which is a limitation of the current approach.

Rendering techniques. The ghost chair with ghost stairs provides immediate spatial context that text-based error codes fundamentally cannot offer. However, the current evaluation of rendering effectiveness is qualitative rather than quantitative. Without a controlled user study comparing diagnosis time and error rate, the claim of "faster and more intuitive" diagnosis remains an inductively supported hypothesis rather than a deductively proven conclusion.

Validation. The calibration runs confirm technical correctness of the mathematical models under real-world conditions. The position accuracy ($<$5 mm on standard rails), jitter suppression (factor $>$20 reduction), and frame rate (60 fps on desktop, 52–60 fps on tablet) meet or exceed the criteria defined in Section 3.6. The main limitation is sample size: three calibration runs on three rail configurations provide evidence of correctness but cannot establish statistical generalizability.

5.2 Answer to Central Question

The data processing pipeline required to transform live stairlift telemetry into a live 3D visualization consists of five layers: wireless transport, checksummed protocol parsing, multi-rate coroutine polling, recursive state estimation, and spline-interpolated 3D rendering. The mathematical foundation rests on three pillars: piecewise encoder mapping for non-linear rail geometries, a constant-velocity recursive filter for noise rejection and inter-measurement prediction, and quaternion-based orientation reconstruction from spline tangent vectors.

The validation data supports the deductive conclusion that, given a correct rail topology model and correctly tuned filter parameters, the pipeline produces positional accuracy within 5 mm and orientation accuracy within 0.5° at horizontal bends. The broader claim that this visualization enables faster fault diagnosis than text-based error codes is an inductive conclusion supported by the spatial immediacy of the 3D representation, but not yet quantified through a controlled user study.

5.3 Limitations

- The validation was performed on three rail configurations. Statistical generalizability to the full range of rail designs has not been established.
- The filter tuning parameters and spindle blend ratio were determined empirically. A formal sensitivity analysis is absent.
- The rendering effectiveness evaluation is qualitative. No controlled user study with measurable diagnosis time reduction has been conducted.
- The arc-tracing approximation introduces a small cumulative error per bend. On rails with many bends ($>$8), this may exceed the 5 mm threshold.

5.4 Recommendations

- **Controlled user study:** Measure diagnosis time and error rate with and without the 3D visualization on a standardized set of fault scenarios to convert the inductive claim into quantitative evidence.
- **Arc-tracing accuracy:** Replace the trapezoidal midpoint approximation with a higher-order numerical integration (e.g., Simpson's rule) to reduce cumulative bend error on long rails.
- **Filter tuning analysis:** Perform a sensitivity analysis on the process and measurement noise parameters to determine safe operating ranges and automate tuning per rail configuration.
- **Historical playback:** Record telemetry sessions for later replay to enable post-incident analysis without a live connection.

Bibliography

[1] Bluetooth SIG. Serial port profile. Adopted Specification v1.2, Bluetooth Special Interest Group, 2022. URL https://www.bluetooth.com/specifications/specs/serial-port-profile-1-2/.

[2] Barry W. Boehm. A spiral model of software development and enhancement. *Computer*, 21(5):61–72, 1988. doi: 10.1109/2.59. URL https://doi.org/10.1109/2.59.

[3] Jherson Bofill, Mideth Abisado, Jocelyn Villaverde, and Gabriel Avelino Sampedro. Exploring digital twin-based fault monitoring: Challenges and opportunities. *Sensors*, 23(16):7087, 2023. doi: 10.3390/s23167087. URL https://www.mdpi.com/1424-8220/23/16/7087.

[4] William Bolton. *Mechatronics: Electronic Control Systems in Mechanical and Electrical Engineering*. Pearson Education, 7th edition, 2019. ISBN 978-1-292-25097-7.

[5] Edwin Catmull and Raphael Rom. A class of local interpolating splines. In R. E. Barnhill and R. F. Riesenfeld, editors, *Computer Aided Geometric Design*, pages 317–326. Academic Press, New York, 1974. doi: 10.1016/B978-0-12-079050-0.50020-5. URL https://doi.org/10.1016/B978-0-12-079050-0.50020-5.

[6] James Diebel. Representing attitude: Euler angles, unit quaternions, and rotation vectors. Technical report, Stanford University, October 2006. URL https://www.astro.rug.nl/software/kapteyn-beta/_downloads/attitude.pdf.

[7] Rory Driscoll. Frame rate independent damping using lerp, 2016. URL https://www.rorydriscoll.com/2016/03/07/frame-rate-independent-damping-using-lerp/. Derives $\mathrm{lerp}(a, b, 1 - e^{-\lambda \Delta t})$ from exponential decay.

[8] Wilfried Elmenreich. An introduction to sensor fusion. *Vienna University of Technology, Austria*, (TR 47/2001), 2002. URL https://www.researchgate.net/publication/267771481_An_Introduction_to_Sensor_Fusion.

[9] Robert David Graham and Peter C. Johnson. Finite state machine parsing for internet protocols: Faster than you think. In *IEEE Symposium on Security and Privacy Workshops*, pages 185–190. IEEE, 2014. doi: 10.1109/SPW.2014.34. URL https://www.ieee-security.org/TC/SPW2014/papers/5103a185.PDF.

[10] Michael Grieves and John Vickers. Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems. In Franz-Josef Kahlen, Shannon Flumerfelt, and Anabela Alves, editors, *Transdisciplinary Perspectives on Complex Systems*, pages 85–113. Springer, Cham, 2017. doi: 10.1007/978-3-319-38756-7_4. URL https://research.fit.edu/media/site-specific/researchfaboraboratories/dmslab/documents/MG--AAIA-Draft2--2016-1-13.pdf.

[11] Rudolph Emil Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME – Journal of Basic Engineering*, 82 (1):35–45, 1960. doi: 10.1115/1.3662552. URL https://www.cs.unc.edu/~welch/kalman/media/pdf/Kalman1960.pdf.

[12] Jack B. Kuipers. *Quaternions and Rotation Sequences: A Primer with Applications to Orbits, Aerospace, and Virtual Reality*. Princeton University Press, 1999. ISBN 978-0-691-05872-6. URL https://press.princeton.edu/books/paperback/9780691102986/quaternions-and-rotation-sequences.

[13] Roger R. Labbe. *Kalman and Bayesian Filters in Python*. Self-published (GitHub), 2015. URL https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python. Chapter 7: Continuous White Noise Model. CC BY 4.0 license.

[14] Juan Linietsky and Ariel Manzur. Godot engine: Multi-platform 2d and 3d open source game engine. 2014. URL https://godotengine.org/. MIT License.

[15] Tomáš Marek and Ondřej Krejcar. Optimization of 3D rendering in mobile devices. In *Mobile Web and Intelligent Information Systems (MobiWIS 2015)*, volume 9228 of *Lecture Notes in Computer Science*, pages 37–48, Cham, 2015. Springer. doi: 10.1007/978-3-319-23144-0_4. URL https://doi.org/10.1007/978-3-319-23144-0_4.

[16] R. J. E. Merry, M. J. G. van de Molengraft, and M. Steinbuch. Velocity and acceleration estimation for optical incremental encoders. *Mechatronics*, 20(1):20–26, 2010. doi: 10.1016/j.mechatronics.2009.06.010. URL https://doi.org/10.1016/j.mechatronics.2009.06.010.

[17] Parag Narkhede, Shashi Poddar, Rahee Walambe, George Ghinea, and Ketan Kotecha. Cascaded complementary filter architecture for sensor fusion in attitude estimation. *arXiv preprint arXiv:2107.03970*, 2021. doi: 10.48550/arXiv.2107.03970. URL https://arxiv.org/abs/2107.03970. Open access, CC BY 4.0.

[18] National Imagery and Mapping Agency. *The American Practical Navigator*. National Imagery and Mapping Agency, Bethesda, MD, bicentennial edition, 2002. URL https://en.wikisource.org/wiki/The_American_Practical_Navigator/Chapter_7. Chapter 7: Dead Reckoning. U.S. Government, public domain.

[19] Les Piegl. On NURBS: A survey. *IEEE Computer Graphics and Applications*, 11(1): 55–71, 1991. doi: 10.1109/38.67702. URL https://fab.cba.mit.edu/classes/S62.12/docs/Piegl_NURBS.pdf.

[20] Ken Shoemake. Animating rotation with quaternion curves. In *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '85)*, pages 245–254. ACM, 1985. doi: 10.1145/325165.325242. URL `https://www.ljll.fr/~frey/cours/references/quaternions.pdf`.

[21] Steven W. Smith. *The Scientist and Engineer's Guide to Digital Signal Processing*. California Technical Publishing, San Diego, CA, 1997. ISBN 0-9660176-3-3. URL `https://www.analog.com/media/en/technical-documentation/dsp-book/dsp_book_ch15.pdf`. Full text available free online.

[22] Gilbert Strang and Edwin Herman. *Calculus Volume 2*. OpenStax, 2016. ISBN 978-1-938168-06-2. URL `https://openstax.org/books/calculus-volume-2/pages/3-6-numerical-integration`. Section 3.6: Numerical Integration. Open access, CC BY-NC-SA 4.0.

[23] Greg Welch and Gary Bishop. An introduction to the Kalman filter. Technical Report TR 95-041, University of North Carolina at Chapel Hill, 1995. URL `https://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf`. Updated 2006.